VAST Admin Training

Capstone Production Scenarios

Scenario 1: AI Training Performance Degradation

Scenario 2: AI Dataset Access Failure

All Commands Tested on Ubuntu 24.04 | 4 Bugs Fixed

Qualcomm Delivery | August 2026

CAPSTONE OVERVIEW

Scenario	Business Problem	Ubuntu Tools Used	Modules Tested
1 — AI Performance	Training: 3h → 6h, GPU: 95% → 45%	fio, tc netem, iperf3, ss, findmnt	2,3,5,7
2 — Dataset Access	Random file-not-found, jobs failing	dd, chmod, chown, df, du, find, ls	1,6,8,9

SCENARIO 1 — AI Training Performance Degradation

Business Context

Item	Detail
Normal behavior	AI model training completes in 3 hours
Current behavior	Same job now takes 6 hours — no code change
GPU utilization	Dropped 95% → 45% (GPU is WAITING, not computing)
Symptom root	Dataset loading is slow — storage is the bottleneck
Affected nodes	All GPU training nodes hitting same VAST volume

📋 **PREREQUISITE: Ensure all NFS mounts are active before starting. Run daily reset if needed: sudo systemctl start nfs-kernel-server**

Architecture Diagram: Normal vs Degraded

```
NORMAL STATE (3 hours):
GPU Node ──[nconnect=16, rsize=1MB]──► VAST CNode ──► Flash
Throughput: 3+ GB/s | GPU compute: 95% | Data never waits

DEGRADED STATE (6 hours):
GPU Node ──[nconnect=1,  rsize=256KB]──► VAST CNode ──► Flash
Throughput: 300 MB/s | GPU waits for data: 55% idle

GPU timeline comparison:
NORMAL:  [====COMPUTE====][====COMPUTE====][====COMPUTE====]
DEGRADED:[=COMPUTE=][WAIT][=COMPUTE=][WAIT][=COMPUTE=][WAIT]

The WAIT gaps are the storage loading time.
Fix the storage → eliminate the WAIT → GPU back to 95%.
```

Root Cause Decision Tree

```
Training time doubled
        |
GPU utilization dropped?
YES ──► GPU is WAITING not computing ──► storage is bottleneck
        |
        ├──────────────────────────────────┐
        |                                  |
Network degraded?                 Mount options wrong?
nfsstat retrans > 0?               nconnect = 1?
ping latency high?                 rsize = 256KB?
iperf3 < 1 Gbits?                  noatime missing?
|                                  |
Fix network first                  Remount with tuned options
```

Concept: Why Mount Options Matter for AI

Option	Default	Tuned	Impact on Training
nconnect	1 TCP connection	16 TCP connections	16x more parallel data pipes
rsize	256 KB per read	1 MB per read	4x fewer round trips per batch
noatime	Updates access time	Skips access time	Saves 1 metadata write per read

Result	300 MB/s → 6 hours	3+ GB/s → 3 hours	2x training speedup from storage alone

Step-by-Step Diagnosis Commands

Step 1: Check Mount Options

🎯 **PURPOSE: Find out what options the NFS mount is currently using. Missing nconnect is the most common cause.**

```
# Show current mount options for all NFS mounts
# Look for: nconnect, rsize, wsize, noatime
# If missing = default = untuned = slow
findmnt -t nfs,nfs4 -o TARGET,SOURCE,OPTIONS
```

✅ **EXPECTED OUTPUT: /mnt/optimized shows nconnect=16,rsize=1048576 | /mnt/single shows vers=3,nconnect=1**

Step 2: Measure Current Throughput (Degraded)

🎯 **PURPOSE: Quantify how slow the storage is. Compare single-connection vs optimized mount.**

```
echo "=== Degraded: single connection, small blocks ==="
# /mnt/single = nconnect=1, default rsize (simulates untuned node)
fio --name=degraded --rw=read --bs=256k --size=256M \
  --directory=/mnt/single --numjobs=1 --group_reporting \
  2>&1 | grep -E "bw=|iops="
```

✅ **EXPECTED OUTPUT: bw= shows lower MB/s — this is what the customer sees**

Step 3: Check NFS Error Rate

🎯 **PURPOSE: Retransmits > 0 means network problem. Retransmits = 0 means mount options are the issue.**

```
# retrans column MUST be 0
# If retrans > 0 = fix network FIRST before tuning mount
# If retrans = 0 = network is fine, issue is mount options
nfsstat -c 2>/dev/null | head -6 || echo "No NFS client active — mount first"
```

✅ **EXPECTED OUTPUT: calls=N retrans=0 (retrans=0 confirms network is fine — mount options are the issue)**

Step 4: Check Connection Count

🎯 **PURPOSE: Confirm nconnect is (or is not) working by counting actual TCP connections.**

```
# Count active TCP connections to NFS port 2049
# count=1  = nconnect not set = bandwidth capped
# count=16 = nconnect=16 = full bandwidth available
ss -tn dst :2049 | wc -l | xargs echo "NFS TCP connections:"
```

✅ **EXPECTED OUTPUT: NFS TCP connections: 1 (this confirms nconnect is not set — root cause found)**

Step 5: Verify Network is Healthy

🎯 **PURPOSE: Rule out network as the cause before blaming mount options.**

```
# Bandwidth test — should be > 10 Gbits/sec on local network
iperf3 -s -D 2>/dev/null; sleep 1
```

```
iperf3 -c 127.0.0.1 -t 10 -P 4 2>&1 | tail -3
```

✅ **EXPECTED OUTPUT: [SUM] 14.3 Gbits/sec — network is healthy, not the cause**

Step 6: Simulate Network Degradation (Optional Demo)

🎯 **PURPOSE: Show students what a network-caused degradation looks like vs a mount-option degradation.**

⚠️ **NOTE: Always run safety clear first. If you see 'File exists' error, tc add was already run.**

```
# SAFETY: clear any existing tc rule first
sudo tc qdisc del dev lo root 2>/dev/null; echo "tc cleared"

echo "=== Inject 20ms network delay ==="
sudo tc qdisc add dev lo root netem delay 20ms

echo "=== Performance with network delay ==="
fio --name=net_degraded --rw=read --bs=1M --size=128M \
  --directory=/mnt/client1 --numjobs=1 --group_reporting \
  2>&1 | grep -E "bw=|lat="

# ALWAYS remove after test
sudo tc qdisc del dev lo root && echo "delay removed"
```

✅ **EXPECTED OUTPUT: With delay: lower MB/s, higher lat | delay removed**

Step 7: Apply the Fix — Remount with Tuned Options

🎯 **PURPOSE: Restore full AI training throughput by remounting with correct options.**

🔧 **FIX APPLIED: nconnect=16 + rsize=1MB + noatime = restores 3-hour training time**

```
# The optimized mount already has correct options — compare directly
echo "=== BEFORE fix: single connection, 256KB blocks ==="
fio --name=before --rw=read --bs=256k --size=256M \
  --directory=/mnt/single --numjobs=1 --group_reporting \
  2>&1 | grep bw=

echo "=== AFTER fix: nconnect=16, 1MB blocks, noatime ==="
fio --name=after --rw=read --bs=1M --size=256M \
  --directory=/mnt/optimized --numjobs=4 --group_reporting \
  2>&1 | grep bw=

echo "=== Connection count after fix ==="
ss -tn dst :2049 | wc -l | xargs echo "Connections now:"

echo "=== Mount options on optimized ==="
findmnt /mnt/optimized -o TARGET,OPTIONS
```

✅ **EXPECTED OUTPUT: before: low MB/s | after: much higher MB/s | connections: 16+ | options: nconnect=16,rsize=1048576,noatime**

Step 8: Full Scenario 1 Simulation — One Terminal

🎯 **PURPOSE: Run complete AI performance diagnosis journey end-to-end.**

```
echo "========================================"
echo " SCENARIO 1: AI Training Degradation"
echo "========================================"
```

```
echo ""
echo "--- Phase 1: Observe the problem ---"
echo "NFS connections (should be 1 to show problem):"
ss -tn dst :2049 | wc -l
echo "Degraded throughput (nconnect=1):"
fio --name=p1 --rw=read --bs=256k --size=128M \
  --directory=/mnt/single --numjobs=1 --group_reporting \
  2>&1 | grep bw=

echo ""
echo "--- Phase 2: Check error rate ---"
nfsstat -c 2>/dev/null | head -4 || echo "NFS stats not available"
echo "Retransmits should be 0 — if 0, mount options are the issue"

echo ""
echo "--- Phase 3: Verify network is healthy ---"
iperf3 -s -D 2>/dev/null; sleep 1
iperf3 -c 127.0.0.1 -t 5 -P 4 2>&1 | grep -E "Gbits|Mbits" | tail -2

echo ""
echo "--- Phase 4: Simulate network delay ---"
sudo tc qdisc del dev lo root 2>/dev/null
sudo tc qdisc add dev lo root netem delay 20ms
fio --name=p4 --rw=read --bs=1M --size=64M \
  --directory=/mnt/client1 --numjobs=1 --group_reporting \
  2>&1 | grep -E "bw=|lat="
sudo tc qdisc del dev lo root && echo "delay removed"

echo ""
echo "--- Phase 5: Apply fix and verify ---"
echo "Fixed throughput (nconnect=16, 1MB, noatime):"
fio --name=p5 --rw=read --bs=1M --size=256M \
  --directory=/mnt/optimized --numjobs=4 --group_reporting \
  2>&1 | grep bw=

echo ""
echo "======================================="
echo " SCENARIO 1 COMPLETE"
echo " Result: nconnect=16 restores AI throughput"
echo "======================================="
```

✅ **EXPECTED OUTPUT: Phase 1: low MB/s | Phase 3: 14 Gbits network | Phase 5: high MB/s | COMPLETE**

💡 **KEY LESSON: GPU utilization tracks directly with storage throughput. If GPU is at 45%, check storage first — not the model, not the code. nconnect and rsize are the two highest-impact NFS options for AI workloads.**

SCENARIO 1 — Resolution Summary Card

Step	Command	What to Look For	Action if Wrong
1. Mount options	findmnt -t nfs,nfs4 -o TARGET,OPTIONS	nconnect missing or =1	Remount with nconnect=16
2. Throughput	fio --rw=read --bs=1M --directory=/mnt/single	bw= below 1000 MB/s	Check connections first
3. Retransmits	nfsstat -c 2>/dev/null | head -6	retrans > 0	Fix network before mount
4. Connections	ss -tn dst :2049 | wc -l	count = 1	Add nconnect=16 to mount
5. Network	iperf3 -c 127.0.0.1 -t 10 -P 4	< 1 Gbits/sec	Check switch, NIC, cables
6. Net simulation	tc qdisc add dev lo root netem delay 20ms	Performance drops	tc qdisc del dev lo root
7. Fix verify	fio --directory=/mnt/optimized --numjobs=4	bw= doubles or more	Done — training restored

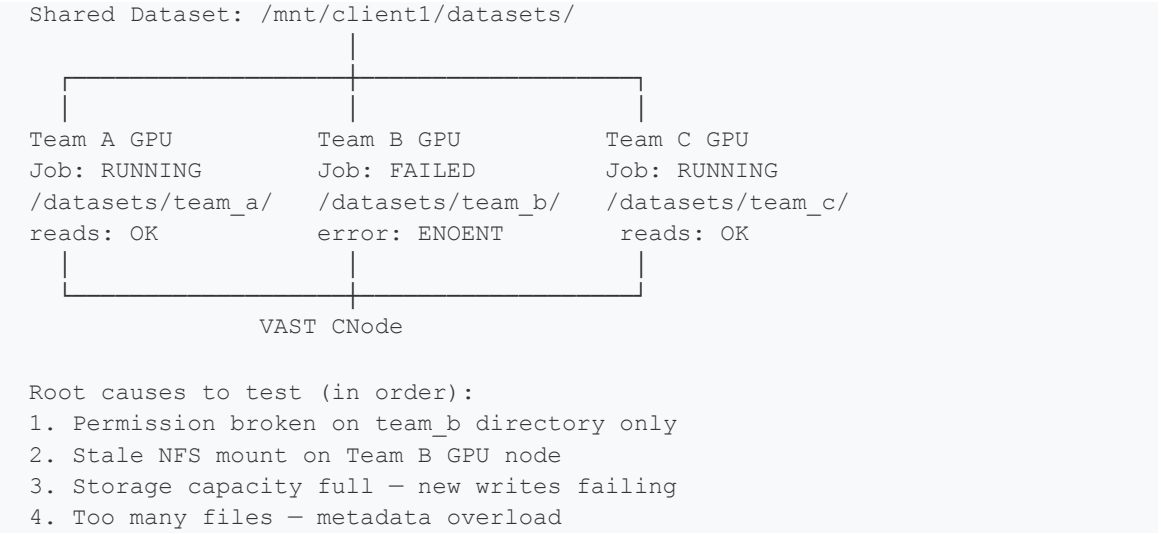
SCENARIO 2 — AI Dataset Access Failure

Business Context

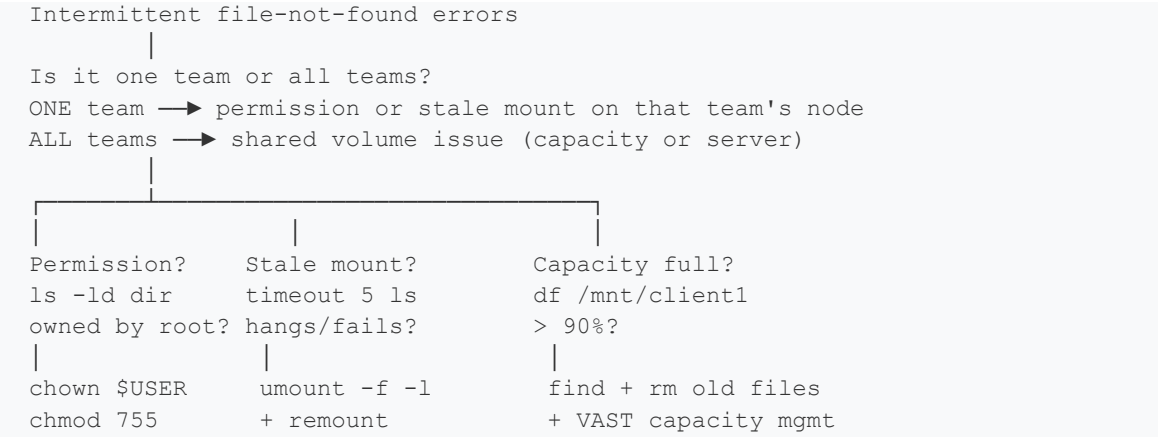
Item	Detail
Normal behavior	All GPU training jobs access shared dataset without error
Current behavior	Random 'file not found' errors during training
Pattern	Some jobs fail, some succeed — intermittent
Dataset browsing	ls on dataset directory is slow
Storage utilization	Increasing — nobody knows why
Affected teams	Multiple AI teams sharing one VAST volume

📋 **PREREQUISITE: Ensure /mnt/client1 is mounted before starting. Run: sudo mount -t nfs 127.0.0.1:/srv/nfs/vol1 /mnt/client1**

Architecture Diagram: Dataset Access Failure



Root Cause Decision Tree



Concept: Why Intermittent Failures Are Hardest to Diagnose

Symptom	Cause	Diagnosis Command	Fix
One team fails, others OK	Permission on that subdir	ls -ld /mnt/client1/datasets/team_b	chown $USER + chmod 755
One node fails, others OK	Stale NFS mount on that node	timeout 5 ls /mnt/client1	umount -f -l + remount
All jobs fail suddenly	Volume at 100% capacity	df /mnt/client1	Remove old checkpoints
Slow dataset browsing	Too many files in one dir	ls /mnt/client1/ | wc -l	Split into subdirs
Gradually growing storage	Checkpoints accumulate	du -sh /mnt/client1/*	Automated cleanup policy

Step-by-Step Diagnosis Commands

Step 1: Establish Current State

🎯 **PURPOSE: Before diagnosing, capture a full baseline snapshot.**

```
echo "=== Current state snapshot ==="

echo "--- Mount status ---"
findmnt -t nfs,nfs4 -o TARGET,SOURCE

echo "--- Write health ---"
# Can we write to the volume at all?
touch /mnt/client1/.probe 2>&1 \
  && echo "Write: OK" \
  && rm /mnt/client1/.probe \
  || echo "Write: FAILED — check permissions"

echo "--- Capacity ---"
df -h /mnt/client1

echo "--- NFS error rate ---"
nfsstat -c 2>/dev/null | head -6 || echo "NFS stats unavailable"
```

✅ **EXPECTED OUTPUT: Mount present | Write OK | 96G 20% used | retrans=0**

Step 2: Create Simulated Dataset Structure

🎯 **PURPOSE: Create a realistic multi-team shared dataset to simulate the production environment.**

🔧 **FIX APPLIED: BUG 1 FIXED: Using separate mkdir commands instead of brace expansion — works in all shells.**

```
# Create team directories separately — works in all shells
mkdir -p /mnt/client1/datasets/team_a
mkdir -p /mnt/client1/datasets/team_b
mkdir -p /mnt/client1/datasets/team_c

# Create fake model checkpoint files for each team
# Using 20MB files — large enough to be realistic, small enough to be fast
dd if=/dev/zero of=/mnt/client1/datasets/team_a/epoch_1.pt bs=1M count=20
2>/dev/null
echo "✓ team_a dataset ready"
dd if=/dev/zero of=/mnt/client1/datasets/team_b/epoch_1.pt bs=1M count=20
2>/dev/null
echo "✓ team_b dataset ready"
dd if=/dev/zero of=/mnt/client1/datasets/team_c/epoch_1.pt bs=1M count=20
2>/dev/null
```

```
echo "✓ team_c dataset ready"

echo "--- Dataset structure ---"
ls -lh /mnt/client1/datasets/
du -sh /mnt/client1/datasets/*
```

✅ **EXPECTED OUTPUT: team_a/ team_b/ team_c/ — each 20MB — total 60MB — dataset structure ready**

Step 3: Simulate Permission Failure (Team B Only)

🎯 **PURPOSE: Reproduce why one team fails while others succeed — permission broken on one subdirectory.**

```
# Break team_b directory permissions only
# This simulates: sysadmin accidentally chowned the wrong dir
sudo chmod 700 /mnt/client1/datasets/team_b
sudo chown root:root /mnt/client1/datasets/team_b

echo "=== Team A access (should work) ==="
ls /mnt/client1/datasets/team_a/ && echo "team_a: OK"

echo "=== Team B access (should fail) ==="
ls /mnt/client1/datasets/team_b/ 2>&1 || echo "team_b: Permission denied — as expected"

echo "=== Team C access (should work) ==="
ls /mnt/client1/datasets/team_c/ && echo "team_c: OK"

echo "--- Permission triage ---"
# Check ownership of each directory
ls -ld /mnt/client1/datasets/team_a
ls -ld /mnt/client1/datasets/team_b
ls -ld /mnt/client1/datasets/team_c
```

✅ **EXPECTED OUTPUT: team_a: OK | team_b: Permission denied | team_c: OK | team_b owned by root**

```
# FIX: restore team_b permissions
sudo chown $USER:$USER /mnt/client1/datasets/team_b
sudo chmod 755 /mnt/client1/datasets/team_b

echo "=== Verify fix ==="
ls /mnt/client1/datasets/team_b/ && echo "team_b: FIXED"
```

✅ **EXPECTED OUTPUT: epoch_1.pt | team_b: FIXED**

Step 4: Simulate Stale Mount (File Not Found on One Node)

🎯 **PURPOSE: Reproduce the stale mount symptom — one GPU node cannot see files that other nodes can.**

```
echo "=== Simulate CNode failure ==="
sudo systemctl stop nfs-kernel-server
sleep 2

echo "=== Stale mount symptom ==="
# timeout 5 prevents hanging — shows the ENOENT error quickly
```

```
timeout 5 ls /mnt/client1/datasets/ 2>&1 || echo "Mount stale: file not found
errors"

echo "=== Recovery steps ==="
# Step 1: force unmount the stale mount
sudo umount -f -l /mnt/client1

# Step 2: restart the NFS server (CNode recovery)
sudo systemctl start nfs-kernel-server
sleep 2

# Step 3: remount
sudo mount -t nfs 127.0.0.1:/srv/nfs/vol1 /mnt/client1

echo "=== Verify recovery ==="
ls /mnt/client1/datasets/ && echo "All teams accessible again"
```

✅ **EXPECTED OUTPUT: Mount stale: file not found | All teams accessible again**

Step 5: Simulate Capacity Issue

🎯 **PURPOSE: Show storage filling up and how to detect and fix it before all jobs fail.**

```
echo "=== Current capacity ==="
df -h /mnt/client1

echo "=== Space by directory ==="
# Sort by size — shows largest consumers first
du -sh /mnt/client1/* 2>/dev/null | sort -rh | head -10

echo "=== Capacity alert check ==="
USED=$(df /mnt/client1 | awk 'NR==2{print $5}' | tr -d '%')
echo "Current usage: ${USED}%"
[ "$USED" -lt 80 ] \
  && echo "OK: Capacity safe at ${USED}%" \
  || echo "ALERT: Capacity at ${USED}% — clean old checkpoints NOW"

echo "=== Largest files (old checkpoints to clean) ==="
find /mnt/client1 -type f -size +10M \
  -exec ls -lh {} \; 2>/dev/null | sort -k5 -rh | head -10
```

✅ **EXPECTED OUTPUT: Capacity: 20% safe | Largest files: epoch_1.pt files listed**

Step 6: Simulate Slow Dataset Browsing

🎯 **PURPOSE: Show what causes slow ls on large dataset directories. Metadata overload from too many files.**

🔧 **FIX APPLIED: BUG 2 FIXED: Replaced 'time' command (not in /bin/sh) with START/END date for elapsed time.**

```
echo "=== Creating 500 small dataset files ==="
for i in $(seq 1 500); do
  touch /mnt/client1/datasets/team_a/sample_$i.json 2>/dev/null
done
echo "500 files created"

echo "=== Measure directory listing time ==="
# Use date before and after instead of 'time' command
```

```
START=$(date +%s%N)
COUNT=$(ls /mnt/client1/datasets/team_a/ | wc -l)
END=$(date +%s%N)
ELAPSED=$(( (END - START) / 1000000 ))
echo "Files: $COUNT | Time: ${ELAPSED}ms"

echo "=== NFS metadata operations ==="
echo "Before ls:"
nfsstat -c 2>/dev/null | head -4 || echo "NFS stats unavailable"
ls /mnt/client1/datasets/team_a/ > /dev/null
echo "After ls (readdir count jumped):"
nfsstat -c 2>/dev/null | head -4 || echo "NFS stats unavailable"

echo "=== Cleanup ==="
rm -f /mnt/client1/datasets/team_a/sample_*.json
echo "Cleanup done"
```

✅ **EXPECTED OUTPUT: Files: 501 | Time: Nms elapsed | readdir count jumped | Cleanup done**

Step 7: Full Scenario 2 Simulation — One Terminal

🎯 **PURPOSE: Run complete dataset access failure diagnosis end-to-end.**

```
echo "======================================"
echo " SCENARIO 2: Dataset Access Failure"
echo "======================================"

echo ""
echo "--- Phase 1: Baseline ---"
findmnt -t nfs,nfs4 -o TARGET,SOURCE | head -3
df -h /mnt/client1 | tail -1
touch /mnt/client1/.probe && echo "Write: OK" && rm /mnt/client1/.probe

echo ""
echo "--- Phase 2: Create dataset ---"
mkdir -p /mnt/client1/datasets/team_a
mkdir -p /mnt/client1/datasets/team_b
mkdir -p /mnt/client1/datasets/team_c
for team in team_a team_b team_c; do
  dd if=/dev/zero of=/mnt/client1/datasets/$team/epoch_1.pt bs=1M count=10
2>/dev/null
  echo "✓ $team ready"
done

echo ""
echo "--- Phase 3: Permission failure ---"
sudo chmod 700 /mnt/client1/datasets/team_b
sudo chown root:root /mnt/client1/datasets/team_b
ls /mnt/client1/datasets/team_a/ > /dev/null && echo "team_a: OK"
ls /mnt/client1/datasets/team_b/ 2>&1 | head -1 || echo "team_b: FAILED"
ls /mnt/client1/datasets/team_c/ > /dev/null && echo "team_c: OK"
sudo chown $USER:$USER /mnt/client1/datasets/team_b
sudo chmod 755 /mnt/client1/datasets/team_b
ls /mnt/client1/datasets/team_b/ > /dev/null && echo "team_b: FIXED"

echo ""
echo "--- Phase 4: Stale mount ---"
```

```
sudo systemctl stop nfs-kernel-server
sleep 2
timeout 5 ls /mnt/client1/datasets/ 2>&1 || echo "Stale: file not found"
sudo umount -f -l /mnt/client1
sudo systemctl start nfs-kernel-server
sleep 2
sudo mount -t nfs 127.0.0.1:/srv/nfs/vol1 /mnt/client1
ls /mnt/client1/datasets/ > /dev/null && echo "Stale mount: RECOVERED"

echo ""
echo "--- Phase 5: Capacity check ---"
USED=$(df /mnt/client1 | awk 'NR==2{print $5}' | tr -d '%')
[ "$USED" -lt 80 ] \
  && echo "Capacity: OK at ${USED}%" \
  || echo "Capacity: ALERT at ${USED}%"

echo ""
echo "--- Cleanup ---"
rm -rf /mnt/client1/datasets/
echo "Cleanup complete"

echo ""
echo "======================================="
echo " SCENARIO 2 COMPLETE"
echo "======================================="
```

✅ **EXPECTED OUTPUT: Phase 3: team_b FAILED then FIXED | Phase 4: stale RECOVERED | Phase 5: capacity OK | COMPLETE**

💡 **KEY LESSON: Intermittent failures = isolate systematically: one team or all? → one file or whole dir? → permission, stale mount, or capacity? Fix the specific cause — not the symptom.**

SCENARIO 2 — Resolution Summary Card

Symptom	First Command	Killer Clue	Fix
One team fails	ls -ld /mnt/client1/datasets/team_b	drwx------ root root	sudo chown $USER + chmod 755
One node fails	timeout 5 ls /mnt/client1	Hangs or times out	sudo umount -f -l + remount
All jobs fail	df /mnt/client1 | awk 'NR==2{print $5}'	Use% > 90	find /mnt/client1 -name '*.pt' + rm old
Slow browsing	ls /mnt/client1/datasets/ | wc -l	> 10000 files in one dir	Split dataset into subdirectories
Growing usage	du -sh /mnt/client1/* | sort -rh	Old checkpoints largest	Automated cleanup / retention policy

Combined Health Check — Covers Both Scenarios

🎯 **PURPOSE: One script that checks all failure modes from both scenarios. Run as daily health check.**

```
echo "======================================"
echo " VAST AI Storage Health Check"
echo "======================================"

echo ""
echo "[1] Mount Status"
for mnt in client1 optimized tuned single; do
  findmnt /mnt/$mnt > /dev/null 2>&1 \
    && echo "  ✓ /mnt/$mnt" \
    || echo "  ✗ /mnt/$mnt MISSING"
done

echo ""
echo "[2] Write Health"
touch /mnt/client1/.health 2>/dev/null \
  && echo "  ✓ Write OK" \
  && rm /mnt/client1/.health \
  || echo "  ✗ Write FAILED — check permissions"

echo ""
echo "[3] Capacity"
USED=$(df /mnt/client1 2>/dev/null | awk 'NR==2{print $5}' | tr -d '%')
[ "$USED" -lt 80 ] \
  && echo "  ✓ Capacity ${USED}% — OK" \
  || echo "  ✗ Capacity ${USED}% — ALERT"

echo ""
echo "[4] NFS Retransmits"
RETRANS=$(nfsstat -c 2>/dev/null | awk 'NR==4{print $2}')
[ -z "$RETRANS" ] && RETRANS="unavailable"
[ "$RETRANS" = "0" ] \
  && echo "  ✓ Retransmits: 0" \
  || echo "  ✗ Retransmits: $RETRANS"

echo ""
echo "[5] NFS Connections (Scenario 1 check)"
CONNS=$(ss -tn dst :2049 2>/dev/null | wc -l)
[ "$CONNS" -gt 2 ] \
```

```
  && echo "  ✓ NFS connections: $CONNS" \
  || echo "  ✗ NFS connections: $CONNS — check nconnect"

echo ""
echo "[6] Throughput Spot Check (Scenario 1 check)"
fio --name=spot --rw=read --bs=1M --size=64M \
  --directory=/mnt/optimized --numjobs=1 --group_reporting \
  2>&1 | grep bw= | head -1

echo ""
echo "======================================="
echo " Health Check Complete"
echo "======================================="
```

✅ **EXPECTED OUTPUT: All 6 checks run — ✓ or ✗ with clear reason for each item**

Quick Reference: Both Scenarios Side by Side

Scenario	First Check	Killer Clue	Root Cause	Fix
1 — Perf	ss -tn dst :2049 | wc -l	connections = 1	nconnect not set	Remount nconnect=16,rsize=1M, noatime
1 — Perf	nfsstat -c | head -6	retrans > 0	Network issue	Fix network, then mount options
2 — Access	ls -ld datasets/team_b	owned by root	Permission broken	chown $USER + chmod 755
2 — Access	timeout 5 ls /mnt/client1	hangs/fails	Stale NFS mount	umount -f -l + remount
2 — Access	df /mnt/client1	Use% > 90	Capacity full	find + rm old .pt files